

Computer Architecture and Technology Area (ARCOS)

Universidad Carlos III de Madrid



OPERATING SYSTEMS

Laboratory 3. Multi-threaded programming: Smart parking

Bachelor's Degree in Computer Science and Engineering

Bachelor's Degree in Mathematics and Computing

**Double Degree in Computer Science and Engineering and Business
Administration**

Course 2025/2026

Table of Contents

1. Statement of the Practice	1
2. Description of the Practice	3
2.1. Parking Manager (parking_manager)	4
2.2. Entry Lane Threads (entry_lane_thread)	5
2.3. Parking Space Control (Lot)	6
2.4. Exit Gate Threads (exit_gate_thread)	6
2.5. Integration and Implementation Examples	6
3. Supporting Source Code	11
4. Compilation and Execution	11
5. Delivery	13
5.1. Submission Deadline	13
5.2. Procedure for Submitting Assignments	13
5.3. Documentation to be Submitted	13
6. Rules	16
7. Bibliography	17

1. Statement of the Practice

This practice allows the student to become familiar with the services for light process management provided by POSIX.

For the management of lightweight processes (threads), system calls **pthread_create**, **pthread_join** and **pthread_exit** will be used, while for their synchronisation, **mutex** and **conditional variables will be used**:

- **pthread_create**: Creates a new thread that executes a function indicated as an argument in the call.
- **pthread_join**: Waits for a thread that needs to end and is indicated as the argument of the call.
- **pthread_exit**: The process making the call completes execution.

The student must design and code, in C language and on the Linux operating system, a program that acts as an intelligent parking lot for the control of parking spaces, including several threads responsible for managing different phases of the parking and a thread that carries out the planning of the phases

2. Description of the Practice

The objective of this practice is to code a simulation of the operation of a smart parking lot. Many of today's parking lots must face two simultaneous challenges: maintaining a real-time count of occupied spaces and applying priority rules that favour certain categories of vehicles. In this practice, the functions of a basic parking lot are extended to a realistic system of priority lanes. To do this, a configuration of **four entrance gates** should be considered, each housing **three independent lanes** with the following priorities:

- 1 - Eco-friendly or zero-emission cars (top priority).
- 2 - Residents
- 3 - General public or visitors (minimum priority).

In addition, it should be considered that there are only **two exit gates**.

This design must be carried out using POSIX threads (*pthreads*) that work concurrently, ensuring correct synchronisation in the shared resources (**through mutex and condition variables**), avoiding race conditions and respecting the priority and maximum capacity rules. To do this, the following roles in the threads must be considered:

- ***parking_manager***: It is the main thread in charge of initialising the resources for the car park, launching the twelve entry threads (4 entrance gates * 3 lanes) and the two exit threads, as well as waiting for them to finish.
- ***entry_lane_thread***: These are the threads in charge of managing the lanes assigned to each of the four entrance gates. Each thread simulates the continuous arrival of vehicles, waits for a free space, acquires control of the barrier according to its priority and increases occupancy.
- ***exit_threads***: These are the threads in charge of managing the two exit gates. It simulates vehicles leaving the car park, reducing occupancy and freeing up a slot.

The design of the system will be based on a **Producer-Consumer model**. The main parking manager (*parking_manager*) will act as a single producer thread that reads vehicle data from a file and inserts it into 12 independent queues (one for each lane). For its part, each of the 12 entry threads (*entry_lane_thread*) will operate as a consumer, sequentially extracting vehicles from their assigned queue and managing their access to the car park.

2.1. Parking Manager (*parking_manager*)

The *parking_manager* must be implemented in the *parking_manager.c* file. In addition to managing, the *parking_manager* will act as a producer, with its main tasks being:

1. Initialise a shared structure called Lot that contains the parking configuration.
2. Initialise 12 circular queues that will contain the records of the vehicles that will access the car park. A circular queue belongs exclusively to one lane of an entrance.
3. Run the threads of the entrance rails (12 threads in total: 4 entrance gates * 3 lanes per gate) and the output threads (2 threads in total). The input threads get stuck waiting for the *parking_manager* signal. Each thread has a circular queue.
4. Read and interpret the input file by copying the data to memory within a structure vector (*struct car*).
5. Read the structure vector generated in the previous step and insert each record into the corresponding circular queue (*queue_put*).
6. Wait for the completion of all threads (*pthread_join*) and release the resources.

The program receives three arguments in the following order:

1. The path to a text file in CSV format (comma-separated values).
2. The size of the circular queues.
3. The parking capacity or number of spaces.

The input CSV file contains: the list of vehicle license plates, the entry gate, and its priority. An example of this input file is shown below:

None

Number Plate, Gate, Priority

1111-AAA, 0, 2

2222-BBB, 1, 2

3333-CCC, 2, 2

4444-DDD, 3, 2

5555-EEE, 0, 2

6666-FFF, 1, 2

7777-GGG, 2, 2

8888-HHH, 3, 2

9999-III, 0, 2

```
1010-JJJ, 1, 2  
2020-KKK, 2, 2  
3030-LLL, 3, 1  
4040-MMM, 0, 0
```

The first line of the file is a header that the program must ignore. In addition, the program must check that each line within the file is valid according to the following characteristics:

1. **Number Plate** is made up of 4 positive whole digits followed by a hyphen and three capital letters:
`<integer><integer><integer><integer>-<letter><letter><letter>`
2. **Gate** is within the valid range [0,3].
3. **Priority** is within the valid range [0,2].

In case a line is invalid, it must be ignored and continue with the next one.

To read the file, system calls (***open***, ***read***, ***close***) must be used, and if any of them fail, a message should be written to the standard error output using “***perror***” and return -1.

2.2. Entry Lane Threads (*entry_lane_thread*)

They manage the entry of vehicles into the parking. In total, 12 entrance threads will be launched (4 entrance gates * 3 lanes per gate). These threads must perform the following operations:

1. It locks until it receives the signal from the *parking_manager* to allow vehicle access.
2. Extract the next vehicle from its assigned queue using the *queue_get()* function. If the queue is empty, it must wait until an element is available.
3. When an element is obtained, the following message is printed: [LANE <number_plate>][GATE <access_gate>] Car <plate>
3. Acquire a parking space considering its priority. In the event that the parking is full, the execution of the thread must be locked. The higher priority lanes must block the lower priority lanes.
4. Update the parking lot occupancy statistics (*lot_claim_space*).
5. Add the vehicle to one of the exit queues (gates). The exit gate assigned to each vehicle is calculated by applying the module operation between its entrance gate and the total number of exit gates:

$exit_gate = access_gate \% num_exit_gates$

If the producer has finished adding items to the queue and the queue is empty, the thread must finish running (*pthread_exit*) with return code 0.

2.3. Parking Space Control (Lot)

Access to the parking spaces is controlled by a shared structure called Lot. The student must implement two functions:

- **lot_claim_space**(Lot *l): controls thread access to parking spaces. If no spaces are available, it should make the thread wait. When at least one space is available, always respecting access priority, it increments the shared counter of the occupied spaces and prints the message to standard output:

[INFO] LOT CLAIMED <occupied spaces>/<number of spaces> BY <number plate>

and continue with the execution of the thread.

- **lot_release_space**(Lot *l): decrements the shared variable of occupied spaces, sends a signal to the waiting entry threads (*entry_lane_thread*) and prints the message:

[INFO] LOT RELEASED <occupied spaces>/<number of spaces> BY <number plate>

2.4. Exit Gate Threads (*exit_gate_thread*)

Simulates the exit of a vehicle from the parking regardless of its entry priority. It is responsible for freeing up a parking space and reducing the current vehicle counter in the parking (by calling the *lot_release_space* function):

1. It gets an element from the circular output queue previously added by *entry_lane_thread*. If the queue is empty, it must wait until an element is available.
2. When an element is obtained, the following message is printed:
[EXIT <exit gate>] CAR <number plate>
3. Then it calls the *lot_release_space* function.
4. Releases the memory allocated to the vehicle.

2.5. Integration and Implementation Examples

An example of the parking execution flow could be the following:

- Execution of the *parking_manager*:

- Create the Lot shared structure that represents the parking parameters and its synchronisation structures for the parking's correct operation.
- Create the circular queues for access (12) and exit (2).
- Deploy the *entry_lane_thread* (12) and *exit_gate_thread* (2) threads.
- Reads the input file passed as a parameter.
- It opens the file, obtains the information, stores it in memory and closes the file.
- It sends a signal to the *entry_lane_thread* threads to start consuming from their circular queue.
- Insert items read from memory into the appropriate circular queues for each vehicle.
- When all threads are finished, the resources are released, and the program is terminated.
- Execution of the *entry_lane_thread*:
 - Obtaining the input parameters.
 - Wait until the *parking_manager* gives the order to start.
 - Allow vehicle access by reading the assigned circular queue.
 - Get a circular queue element.
 - Increases the counter of occupied spaces within the shared Lot structure by considering the priority of the vehicle based on the element obtained (*lot_claim_space*).
 - Add the vehicle to an exit gate (circular exit queue).
 - When the threads finish, the resources are released, and their execution ends.
- Execution of the *exit_gate_thread*:
 - Obtaining the input parameters.
 - Allow vehicles to exit by reading from the assigned circular exit queue.
 - Get an item from the circular output queue.
 - Decrement of the counter of occupied places within the shared structure Lot (*lot_release_space*).
 - When the threads finish, the resources are released, and their execution finishes.

To ensure the program operates correctly, it must print the following traces (**via standard output unless otherwise specified**) at each specified time. To do this, use the `printf` or `fprintf` function, as appropriate, adding a newline character (`\n`) at the end of each message. It is **mandatory** to follow the specified format.

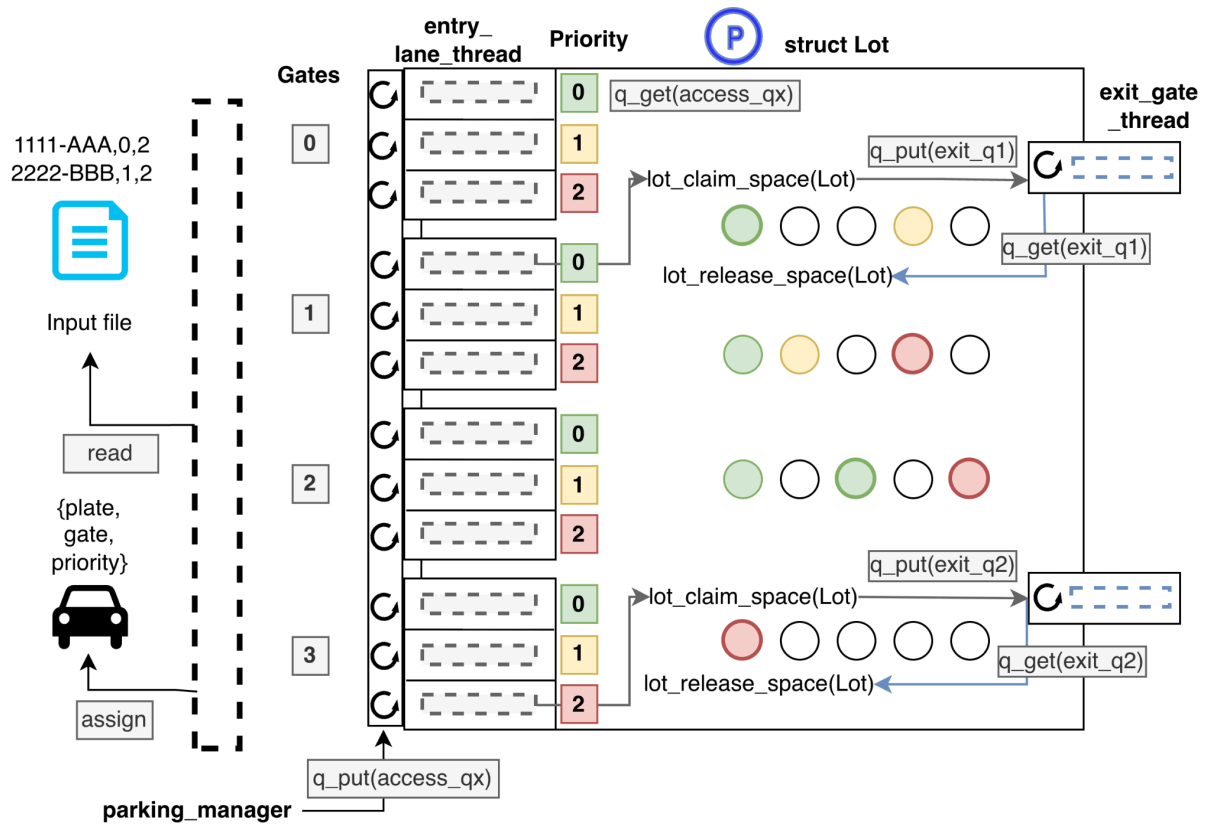
- *parking_manager*:

- When an error occurs in opening, reading, closing, or parsing the input file, any message with **perror** should be returned through **standard error output** and the value -1 should be returned.
- When an *entry_lane_thread* finishes with errors, it should use the error output **without terminating the main program execution**:
 - “[ERROR][parking_manager] LANE <lane_number> of GATE <access_gate> FINISHED WITH ERROR”
 - `fprintf(stderr, "[ERROR][parking_manager] LANE %d of GATE %d FINISHED WITH ERROR\n", var_lane, var_gate);`
- When an *exit_gate_thread* finishes with errors, it should use the error output **without terminating the main program execution**:
 - “[ERROR][parking_manager] EXIT GATE <exit_gate> FINISHED WITH ERROR”
 - `fprintf(stderr, "[ERROR][parking_manager] EXIT GATE %d FINISHED WITH ERROR\n", var_gate);`
- Before the program ends:
 - “[OK][parking_manager] Finishing”
 - `printf("[OK][parking_manager] Finishing\n");`
- *entry_lane_thread*:
 - When an error occurs reading the arguments, the **error output is used** and -1 is returned:
 - “[ERROR][entry_lane_thread] Arguments not valid.”
 - `fprintf(stderr, "[ERROR][entry_lane_thread] Arguments not valid\n");`
 - When the arguments have been processed:
 - “[LANE <number_plate>][GATE <access_gate>] Waiting”
 - `printf("[LANE %d][GATE %d] Waiting\n", var_lane, var_gate);`
 - After retrieving an element from the access queue:
 - “[LANE <number_plate>][GATE <access_gate>] Car <plate>”
 - `printf("[LANE %d][GATE %d] Car %s\n", var_lane, var_gate, plate);`
 - Before finishing:
 - “[LANE <number_plate>][GATE <access_gate>] Waiting”
 - `printf("[LANE %d][GATE %d] Ending\n", var_lane, var_gate);`

- *exit_gate_thread*:
 - When an error occurs while reading the arguments, use the **error output** and return -1:
 - “[ERROR][EXIT <exit_gate>] Arguments not valid”
 - `fprintf(stderr, “[ERROR][EXIT %d] Arguments not valid\n”, exit_gate);`
 - After retrieving an element from the exit queue:
 - “EXIT <exit_gate>] CAR <number_plate>”
 - `printf(“[EXIT %d] CAR %s\n”, exit_gate, plate);`
 - Before finishing:
 - “[EXIT <exit_gate>] Ending”
 - `printf(“[EXIT %d] Ending\n”, exit_gate);`
- *lot_claim_space*
 - To increase the number of occupied slots:
 - “[INFO] LOT CLAIMED <occupied slots>/<number of slots> BY <number plate>”
 - `printf(“[INFO] LOT CLAIMED %d/%d BY %s\n”, occupied_spaces, total_spaces, plate);`
- *lot_release_space*
 - After reducing the number of occupied parking spaces:
 - “[INFO] LOT RELEASED <occupied slots>/<number of slots> BY <number plate>”
 - `printf(“[INFO] LOT RELEASED %d/%d BY %s\n”, occupied_spaces, total_spaces, plate);`

In general, if the program detects an error, it should return -1 and write to the standard error output. If the execution is successful, it should end with a return 0.

The figure below illustrates the life cycle of vehicles and the interaction between the simulator threads. The process starts with the **parking_manager**, which reads the input file and assigns vehicles to the entrance queues based on their gate and priority via **queue_put**. Entry threads (**entry_lane_thread**) then pull cars out of these queues with **queue_get** and reserve a parking space using **lot_claim_space**. After their stay, the vehicles go to the exit queues (with **queue_put**), where the corresponding threads (**exit_gate_thread**) process them to finally free up the parking space by **lot_release_space**, thus completing the flow of the system.



3. Supporting Source Code

To facilitate the performance of this practice, the *p3_parking.zip file is available*, which contains supporting source code and can be downloaded from Aula Global. To extract its contents, the following must be executed:

Shell

```
unzip p3_parking.zip
```

By extracting its contents, the directory **p3_parking/** where the practice is to be developed is created. The following files are included within this directory:

Makefile

Source file for the make tool. It should not be modified. With it, automatic compilation is achieved only for the source files that are modified.

parking_manager.c

C source file. This file is the one that must be modified for the development of the command processor.

queue.c

C source file. This file must be modified to define the behaviour of the circular queue.

queue.h

Header file. This file contains the declaration of the functions used in the circular queue.

autores.txt

Text file (.txt) containing the authors of the assignment (one per line).

4. Compilation and Execution

To compile and run the lab, it is **mandatory** to use the "make" command. The initial rules are located in the Makefile provided.

Example compilation:

```
Shell  
make clean  
make
```

Example execution:

```
Shell  
./parking <input file> <queue size> <parking capacity>
```

Students are provided with a Python (version 3) script called "checker_ssou_p3.py" that verifies that the format of the lab assignment is correct. The script checks that the task meets the naming specifications, is properly compressed, includes everything specified, and compiles correctly.

To run the checker, the following command must be used:

```
Shell  
python3 checker_ssou_p3.py <your_zip_file.zip>
```

The file to be delivered via Aula Global is entregable.zip (see the [Delivery](#) section). For example:

```
Shell  
python3 checker_ssou_p3.py ssou_p3_100254896_100047014.zip
```

The checker will print messages on the screen indicating whether the format is correct or not.

5. Delivery

5.1. Submission Deadline

The deadline for submission of the internship in AULA GLOBAL will be **May 3, 2026 (until 11:55 p.m.)**.

5.2. Procedure for Submitting Assignments

The assignments must be submitted electronically. Links will be provided in AULA GLOBAL through which you can submit your assignments. Specifically, a delivery system will be enabled for the practice code and another of the TURNITIN type for the practice report.

5.3. Documentation to be Submitted

In the **deliverer for the code**, a compressed file in zip format must be delivered with the name `ssoo_p3_AAAAAAAAAA_BBBBBBBBBB.zip` where A... A and B... B are the NIAs of the members of the group. The file must contain:

- **parking_manager.c**
- **Makefile**
- **queue.h**
- **queue.c**
- **autores.txt** - CSV text file with one author per line. Format (without accents):
NIA1, Surname(s),First name(s)
NIA2, Surname(s),First name(s)

Textproto

Valid example:

```
100499455,Surname1 Surname2,FirstName  
100499455,Surname1 Surname2,Compound Name
```

Textproto

Example **NO** Valid:

```
,Surname1 Surname2,Firstname1 (does not include the NIA)
(blank line)
Last Name 1, Last Name 2, Compound Name,100499455
499455,Surname1 Surname2,First Name (Incomplete NIA)
```

In the **TURNITIN** deliverer, the file must be submitted with the following name:

ssoo_p3_NIA1_NIA2_NIA3.pdf

The report must contain at least the following sections:

- **Cover page:** with the full names of the authors, NIAs, group to which they belong and email addresses.
- **Table of contents:** with navigation options to the sections indicated by the titles.
- **Description of the code** detailing the main functions implemented. **DO NOT include the source code** of the practice in this section. Any code will be automatically ignored.
- **Battery of tests** used and results obtained. Higher scores will be given to advanced tests, extreme cases, and, in general, to those tests that guarantee the correct functioning of the practice in all cases. It should be taken into account:
 - A program compiling correctly and without warnings does not guarantee that it will work correctly.
 - Avoid duplicate tests that evaluate the same program flows. The score in this section is not measured according to the number of tests, but the degree of coverage of them. It is better to have a few tests that evaluate different cases than many tests that always evaluate the same case.
 - **Avoid** using **screenshots** of code.
- **Conclusions**, problems encountered, how they have been solved and personal opinions.

The following aspects related to the presentation of the assignment will also be scored:

- The report must have page numbers on all pages (except the cover page).
- The text of the report must be justified.

The length of the report must not exceed 15 pages (cover and index included). It is essential to pass the report in order to pass the practice, so you should not neglect its quality.

NOTE: The **report deliverer** allows only one submission. The grade for this submission is the only valid and final one.

Considerations on the use of AI: If AI tools are used and the solutions from multiple groups are the same, it will be considered plagiarism. It is important to remember that this course includes a midterm and a final exam that require students to write programs in C. Without sufficient proficiency, students will not be able to complete these exams successfully.

6. Rules

- 1) Laboratory assignments that do not compile or that do not conform to the functionality and requirements set will receive a score of 0.
- 2) Programs that use library functions (fopen, fread, fwrite, fclose, etc.) or similar, instead of system calls, will receive a grade of 0. The use of statements or functions such as goto is also prohibited.
- 3) If the report is not submitted, the laboratory assignment will receive a grade of 0.
- 4) Special attention will be paid to detecting copied functionalities between two assignments. If common implementations are found in two assignments, the students involved (both those whose work was copied and those who copied it) will lose the grades they earned for continuous assessment.
- 5) The programs should compile without warnings.
- 6) The programs must run on a Linux system; the exercise cannot be performed on other systems (such as Windows or macOS). Furthermore, to ensure the exercise functions correctly, you must verify its compilation and execution on the Guernika server (<https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/>). If the submitted code does not compile or run on this platform, the implementation will not be considered correct.
- 7) An uncommented program will receive a very low grade.
- 8) The assignment will be submitted through Aula Global, as detailed in the section [Delivery](#) of this document. Delivery via email is not permitted without prior authorization.
- 9) The input and output format indicated in each program to be implemented must be respected at all times.
- 10) Error checking must be performed on each program, more thoroughly than explicitly requested in each section.

Submitted programs that do not comply with these standards will not be considered approved.

7. Bibliography

- El lenguaje de programación C: diseño e implementación de programas Félix García, Jesús Carretero, Javier Fernández y Alejandro Calderón. Prentice-Hall, 2002.
- The UNIX System S.R. Bourne Addison-Wesley, 1983.
- Advanced UNIX Programming M.J. Rochkind Prentice-Hall, 1985.
- Sistemas Operativos: Una visión aplicada. Jesús Carretero, Félix García, Pedro de Miguel y Fernando Pérez. McGraw-Hill, 2001.
- Programming Utilities and Libraries SUN Microsystems, 1990.
- Unix man pages (man function)